



Same Parent Domain Authentication Flow

A practical integration document for connecting new Next.js/static projects to the existing AtomX access portal, Google login, workspace selection, and selected-session API flow.

PORTAL

`portal.atomx.in`

PROJECT DOMAINS

`*.atomx.in`

API

`dapi-uat.atomx.in`

Core decision: all projects live under the same parent domain, for example `portal.atomx.in`, `dashboard.atomx.in`, `tag-series.atomx.in`, and `new-project.atomx.in`. This allows the backend to use parent-domain cookies for the selected workspace session.

0. Which flow to implement (read first)

Implement the token handoff (sections 7-11). That is the flow AtomX runs in production today on `dashboard.atomx.in` and `tag-series.atomx.in`. The parent-domain cookie flow described in section 4 is the future direction and is **not usable yet** - it needs backend endpoints that do not exist. Do not build an auth guard around `/auth/me`.

Verified current state

THING	STATUS	WHAT IT MEANS FOR YOU
<code>?token=</code> handoff from portal	Works in production	Build on this.
Session cookie on API calls	Works (verified with Bearer disabled)	Send <code>credentials: "include"</code> as well, it costs nothing.
<code>GET /auth/me</code>	Does not exist	Check the session by decoding the stored token's <code>exp</code> .
<code>POST /auth/logout</code>	Does not exist	Log out by clearing local storage and returning to the portal.
Portal reads <code>?service=</code>	Not implemented	Your project cannot force a service; the user picks the card.
Portal honours <code>?returnTo=</code>	Reauth only	See section 12 before relying on it.

1. Objective

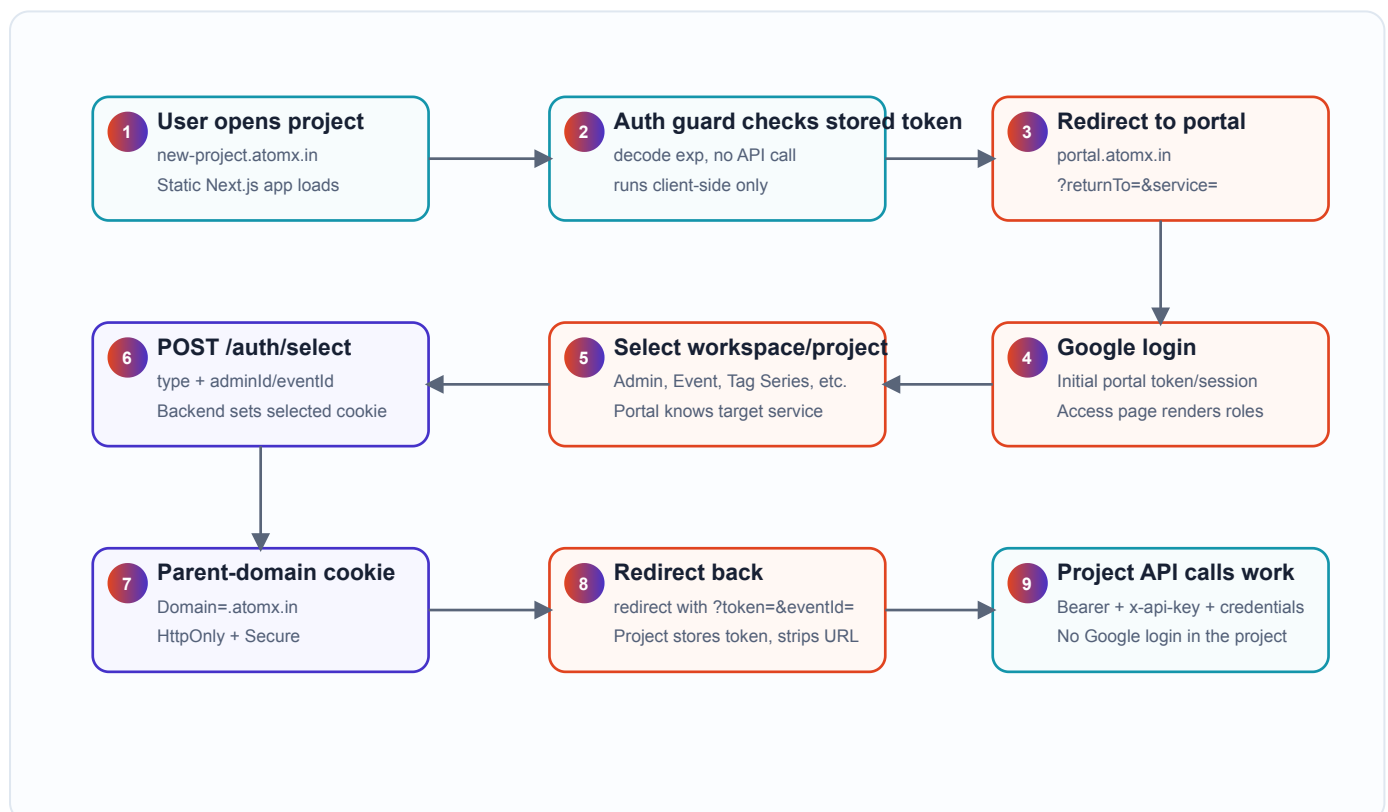
The goal is to let any AtomX project use the same login and access-selection process without rebuilding Google login in every project. A user can open any project subdomain, be redirected to `portal.atomx.in` for authentication, select the correct workspace/project, and return to the project with an active selected session.

The new project can be created by another worker, built as an `out/` static folder, and hosted on a different subdomain, as long as the parent domain remains `atomx.in`.

2. Domain Model

SERVICE	EXAMPLE	RESPONSIBILITY
Access Portal	<code>portal.atomx.in</code>	Google login, access page, workspace/project selection, <code>/auth/select</code> , redirect back to project.
Dashboard	<code>dashboard.atomx.in</code>	Admin/event list, configuration, operations, vendor/stall/device/menu screens.
New Project	<code>new-project.atomx.in</code>	Own UI and business logic; no Google login implementation; uses AtomX session and API client.
Backend API	<code>dapi-uat.atomx.in</code>	Google auth callback, <code>/auth/select</code> , session cookie, protected APIs.

3. Full Flow Diagram



4. Token and Cookie Flow

There are two different auth moments: the initial login session and the selected workspace/project session. The selected session is the one that should authorize project APIs.

1. **Google login:** user authenticates once through `portal.atomx.in`.
2. **Initial token/session:** backend lets the portal render the access page and available roles.
3. **Workspace selection:** portal sends `POST /auth/select` with the selected role context.
4. **Selected session:** backend updates the session cookie to contain selected context such as `adminId`, `eventId`, or `type`.
5. **Project APIs:** the destination project sends API calls with `credentials: "include"`; the backend reads the selected cookie.

Important: if the cookie is `HttpOnly`, frontend code cannot read it with `document.cookie`. That is intended. A project would then have to validate the session through a server endpoint – but `/auth/me` **does not exist on the AtomX backend today**, so this whole section is future work. Until it ships, validate the session by decoding the token you received from the portal (section 9).

Required Backend Cookie Settings

```
Domain=.atomx.in
Path=/
Secure
HttpOnly
SameSite=None
```

Required CORS Settings

The API must allow credentialed requests from all approved project subdomains. Do not use wildcard CORS with credentials.

```
Allowed origins:
https://portal.atomx.in
https://dashboard.atomx.in
https://tag-series.atomx.in
https://new-project.atomx.in

Allow credentials: true
```

5. Redirect Between Subdomains

A new project should redirect unauthenticated users to the portal with a `returnTo` parameter. The portal should preserve this return URL through login and workspace selection.

Project to Portal

```
https://portal.atomx.in?returnTo=https%3A%2F%2Fnew-project.atomx.in%2F&service=new-project
```

Portal Back to Project

```
https://new-project.atomx.in/
```

Because all apps share `.atomx.in`, the selected cookie set by the backend is available to the project subdomain when the project calls the API.

6. New Project Implementation Guide

The integration kit is five small files. The project needs no Google login code, no session endpoint, and no server runtime.

FILE	SECTION	PURPOSE
<code>src/lib/atomxAuthConfig.js</code>	7	Portal URL, API URL, api key, service key, storage keys.
<code>src/lib/atomxToken.js</code>	8	Capture <code>?token=</code> , store/read/clear it, decode <code>exp</code> , read event context.
<code>src/lib/atomxApiClient.js</code>	9	Central fetch: Bearer + <code>x-api-key</code> + <code>credentials: "include"</code> .
<code>src/lib/atomxRedirect.js</code>	10	Store resume context and send the user to the portal.
<code>src/components/AtomxAuthGuard.jsx</code>	11	Client-side gate: capture token, check expiry, redirect, schedule expiry.

7. Code: Auth Config

One file holds every environment-specific value. `service` is the role type the backend registered for this project - the portal uses it to build the storage key.

```
// src/lib/atomxAuthConfig.js
export const ATOMX_AUTH = {
  portalUrl: process.env.NEXT_PUBLIC_PORTAL_URL || "https://portal.atomx.in",
  apiBaseUrl: process.env.NEXT_PUBLIC_BASE_URL || "https://dapi-uat.atomx.in",
  apiKey: process.env.NEXT_PUBLIC_API_KEY || "", // x-api-key, ask the portal owner
  service: "new-project", // role type for this project

  // Storage keys. The portal writes the first one; keep the names exactly.
  tokenKey: "atomx.auth.new-project", // atomx.auth.<service>
  reauthKey: "atomx.portal.reauth", // resume context, 24h TTL
  themeKey: "atomx.theme"
};
```

8. Code: Token Utilities

Decode the JWT locally to read `exp` and the user profile. No library needed - the payload is base64url JSON. Never trust it for authorisation, only for UI and expiry.

```
// src/lib/atomxToken.js
import { ATOMX_AUTH } from "./atomxAuthConfig";

export function decodeJwt(token) {
  try {
    const payload = token.split(".")[1];
    const json = atob(payload.replace(/-/g, "+").replace(/_/g, "/"));
    return JSON.parse(decodeURIComponent(escape(json)));
  } catch {
    return null;
  }
}

export function readToken() {
  if (typeof window === "undefined") return null;
  try {
    return window.localStorage.getItem(ATOMX_AUTH.tokenKey);
  } catch {
    return null;
  }
}

export function storeToken(token) {
  try {
    window.localStorage.setItem(ATOMX_AUTH.tokenKey, token);
  } catch (error) {
    console.error("Failed to store AtomX token", error);
  }
}

export function clearToken() {
  try {
    window.localStorage.removeItem(ATOMX_AUTH.tokenKey);
    window.localStorage.removeItem(ATOMX_AUTH.reauthKey);
  } catch (error) {
    console.error("Failed to clear AtomX token", error);
  }
}

// Capture ?token= handed over by the portal, then remove it from the URL so it
// does not stay in history or get copied into a shared link.
export function captureTokenFromUrl() {
  if (typeof window === "undefined") return null;
  const url = new URL(window.location.href);
  const token = url.searchParams.get("token");
  if (!token) return null;

  storeToken(token);
  url.searchParams.delete("token");
  window.history.replaceState(window.history.state, "", url.toString());
  return token;
}

export function isExpired(token, skewMs = 0) {
  const decoded = decodeJwt(token);
  if (!decoded?.exp) return false; // no exp: let the API decide
  return decoded.exp * 1000 - skewMs <= Date.now();
}

// Event context also arrives as query params on the handoff.
```

```
export function captureContextFromUrl() {
  if (typeof window === "undefined") return {};
  const params = new URLSearchParams(window.location.search);
  return {
    eventId:    params.get("eventId"),
    eventName:  params.get("eventName"),
    venue:      params.get("venue"),
    city:       params.get("city"),
    service:    params.get("service")
  };
}
```

9. Code: API Client

Every AtomX call goes through this helper. It sends the Bearer token, the `x-api-key`, and `credentials: "include"` so the cookie session is used too when it is available.

```
// src/lib/atomxApiClient.js
import { ATOMX_AUTH } from "../atomxAuthConfig";
import { readToken } from "../atomxToken";

export class ApiError extends Error {
  constructor(message, status) {
    super(message);
    this.name = "ApiError";
    this.status = status;
  }
}

export async function atomxFetch(path, options = {}) {
  const url = path.startsWith("http") ? path : `${ATOMX_AUTH.apiUrl}${path}`;
  const token = readToken();
  const isJsonBody = options.body && typeof options.body !== "string";

  const headers = new Headers(options.headers || {});
  headers.set("Accept", "application/json");
  if (isJsonBody) headers.set("Content-Type", "application/json");
  if (ATOMX_AUTH.apiKey) headers.set("x-api-key", ATOMX_AUTH.apiKey);
  if (token) headers.set("Authorization", `Bearer ${token}`);

  const response = await fetch(url, {
    ...options,
    headers,
    credentials: "include",
    cache: options.method && options.method !== "GET" ? undefined : "no-store",
    body: isJsonBody ? JSON.stringify(options.body) : options.body
  });

  if (!response.ok) {
    let message = `Request failed (${response.status})`;
    try {
      const data = await response.json();
      message = data?.message || message;
    } catch {
      /* keep the default message */
    }
    throw new ApiError(message, response.status);
  }

  return response.status === 204 ? null : response.json();
}
```

10. Code: Redirect Helper

Send unauthenticated users to the portal. Store a resume context first so the portal can bring them back to the exact page after they pick their access.

```
// src/lib/atomxRedirect.js
import { ATOMX_AUTH } from "./atomxAuthConfig";
import { decodeJwt, readToken } from "./atomxToken";

const REAUTH_TTL_MS = 24 * 60 * 60 * 1000;

function sanitizeReturnTo(href) {
  const url = new URL(href, window.location.origin);
  url.searchParams.delete("token"); // never round-trip a token
  url.searchParams.delete("returnTo");
  return url.toString();
}

export function redirectToAtomxPortal() {
  if (typeof window === "undefined") return;

  const returnTo = sanitizeReturnTo(window.location.href);
  const profile = decodeJwt(readToken() || "");

  // The portal reads this to resume the same role automatically.
  try {
    window.localStorage.setItem(
      ATOMX_AUTH.reauthKey,
      JSON.stringify({
        version: 1,
        createdAt: Date.now(),
        expiresAt: Date.now() + REAUTH_TTL_MS,
        returnTo,
        type: profile?.ctx?.type || ATOMX_AUTH.service,
        service: ATOMX_AUTH.service,
        eventId: profile?.ctx?.eventId ?? null,
        adminId: profile?.ctx?.adminId ?? null
      })
    );
  } catch (error) {
    console.error("Failed to store resume context", error);
  }

  const url = new URL(ATOMX_AUTH.portalUrl);
  url.searchParams.set("returnTo", returnTo);
  url.searchParams.set("service", ATOMX_AUTH.service);
  window.location.assign(url.toString());
}
```

11. Code: Auth Guard

Wrap the app. The guard captures a handed-over token, then checks expiry. There is no session API to call, so this is entirely local and instant.

```
// src/components/AtomxAuthGuard.jsx
"use client";

import { useEffect, useState } from "react";
import {
  captureTokenFromUrl,
  clearToken,
  decodeJwt,
  isExpired,
  readToken
} from "../lib/atomxToken";
import { redirectToAtomxPortal } from "../lib/atomxRedirect";

const WARN_BEFORE_MS = 10 * 60 * 1000; // match the other AtomX apps

export default function AtomxAuthGuard({ children }) {
  const [status, setStatus] = useState("checking");

  useEffect(() => {
    const token = captureTokenFromUrl() || readToken();

    if (!token || isExpired(token)) {
      clearToken();
      setStatus("redirecting");
      redirectToAtomxPortal();
      return undefined;
    }

    setStatus("ready");

    // Expire in place rather than waiting for a failed request.
    const decoded = decodeJwt(token);
    if (!decoded?.exp) return undefined;

    const msLeft = decoded.exp * 1000 - Date.now();
    const warnAt = setTimeout(() => {
      console.warn("AtomX session expires soon");
    }, Math.max(msLeft - WARN_BEFORE_MS, 0));
    const expireAt = setTimeout(() => {
      clearToken();
      redirectToAtomxPortal();
    }, Math.max(msLeft, 0));

    return () => {
      clearTimeout(warnAt);
      clearTimeout(expireAt);
    };
  }, []);

  if (status !== "ready") return null; // or an AtomX loader
  return children;
}
```

Logout

There is no `/auth/logout`. Clear local storage and return the user to the portal. Any cookie session stays alive until the backend expires it - flag this to the portal owner if hard logout matters for your project.

```
// src/lib/atomxLogout.js
import { ATOMX_AUTH } from "./atomxAuthConfig";
import { clearToken } from "./atomxToken";

export function atomxLogout() {
  clearToken();
  try {
    for (let i = window.localStorage.length - 1; i >= 0; i -= 1) {
      const key = window.localStorage.key(i);
      if (key && key.startsWith("atomx.auth.")) window.localStorage.removeItem(key);
    }
    window.sessionStorage?.clear();
  } catch (error) {
    console.error("Failed to clear AtomX storage", error);
  }
  window.location.assign(`${ATOMX_AUTH.portalUrl}/access/`);
}
```

11b. Where to Use the Guard

Next.js App Router

```
// src/app/layout.jsx
import AtomxAuthGuard from "../components/AtomxAuthGuard";

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <AtomxAuthGuard>{children}</AtomxAuthGuard>
      </body>
    </html>
  );
}
```

Next.js Pages Router

```
// src/pages/_app.jsx
import AtomxAuthGuard from "../components/AtomxAuthGuard";

export default function App({ Component, pageProps }) {
  return (
    <AtomxAuthGuard>
      <Component {...pageProps} />
    </AtomxAuthGuard>
  );
}
```

Reference implementations. The same logic, in production, lives in the monorepo at `apps/dashboard/src/app/components/Session/SessionGuard.js` (token capture, expiry, cross-tab sync) and `apps/dashboard/src/lib/apiClient.js` (headers, error normalisation). Copy from those if anything here is ambiguous.

12. Access Portal Responsibilities

Current behaviour differs from the plan. The portal today **ignores** `?service=`, and it only acts on `?returnTo=` when a resume context (`atomx.portal.reauth`) is present. Arriving with just `returnTo` and no context, the portal redirects straight back to the project *without* a workspace selection - which loops if the project then redirects again. Storing the resume context (section 10) is what avoids this.

What the portal does today

- Completes Google login if there is no portal session.
- Renders access cards from the roles in the decoded token.
- On selection, calls `POST /auth/select`.
- Writes the returned token to `atomx.auth.<service>`.
- Redirects to the service destination with `?token=` plus event context.
- With a valid resume context, repeats the last selection and returns to `returnTo`.

What the portal owner must add for a new project

ITEM	WHERE	WHY
Role type for the project	Backend <code>roles[]</code>	Without it no access card appears and <code>/auth/select</code> has nothing to select. Blocking.
<code>NEXT_PUBLIC_NEWPROJECT_URL</code>	Monorepo root <code>.env</code>	Destination for the redirect. Blocking.
Entry in <code>moduleLinks</code> / <code>MODULE_CATALOG</code>	<code>apps/access_portal/pages/access.js</code>	Card title, description, icon, destination.
<code>x-api-key</code> value	Given to the project	Required header on every API call.
Project origin in CORS allowlist	Backend	Credentialed requests need each explicit origin.

Portal Selection Request

```
POST https://dapi-uat.atomx.in/auth/select
credentials: include
body:
{
  "type": "cashless",
  "eventId": 1840
}

or:
{
  "type": "admin",
  "adminId": 45
}
```

13. API Call Pattern Inside New Project

All project API calls must use the shared helper or at least include `credentials: "include"`.

```
import { atomxFetch } from "../lib/atomxApiClient";

export async function loadEvents() {
  return atomxFetch("/v1/Events/List");
}

export async function loadStalls(eventId) {
  return atomxFetch(`/v1/Stalls/List/Eventwise/${eventId}`);
}
```

Do not build Google login in the new project. The new project should only check session, redirect to portal when needed, and use AtomX APIs with credentials.

14. Static Export Notes

If the new project is exported as a static `out/` folder, the auth guard must run on the client side. This is expected. The static app cannot know the session at build time.

- Use a client component for auth check.
- Do not call protected APIs during static generation.
- Show a loader while checking session.
- If unauthorized, redirect to `portal.atomx.in`.
- After returning, API calls work through the parent-domain cookie.

15. Security Checklist

- Use `HttpOnly` cookies for real session tokens.
- Use `Secure` cookies in production.
- Use `SameSite=None` for cross-subdomain credential requests when needed.
- Set cookie `Domain=.atomx.in`.
- Sanitize `returnTo`; allow only approved `*.atomx.in` destinations.
- Do not put long-lived secrets in query params.
- Allow only known project subdomains in backend CORS.
- Clear stored tokens on logout. **Note:** there is no `/auth/logout`, so a cookie session survives a frontend logout until the backend expires it.
- Strip `?token=` from the URL immediately after capture so it does not persist in history.
- Treat the decoded JWT as UI data only; the backend re-validates every request.

16. Final Worker Handoff Summary

Give this list to whoever builds the new project.

- Do **not** implement Google login, and do **not** call `/auth/me` - it does not exist.
- Add the four files from sections 7-10: auth config, token utils, API client, redirect helper.
- Wrap the app in `AtomxAuthGuard` (section 11).
- On load: capture `?token=` from the URL, store it, strip it from the URL.
- No token or an expired one: store the resume context and redirect to `portal.atomx.in?returnTo=<currentUrl>&service=<projectKey>`.
- Use `atomxFetch` for every AtomX call - it adds Bearer, `x-api-key` and `credentials: "include"`.
- Read event context (`eventId` , `eventName` , `venue` , `city`) from the handoff query params.
- Never call a protected API during static generation; the guard runs client-side only.
- Build normally and deploy the `out/` folder to the project subdomain.

Ask the portal owner for these before you start

- The **role type** registered for this project (used as `service`).
- The **x-api-key** value and the API base URL for your environment.
- Confirmation that the project subdomain is in the backend CORS allowlist.
- Confirmation that `NEXT_PUBLIC_<PROJECT>_URL` and the portal card entry are in place.

Definition of done: opening the project with no session lands on the portal; picking the project's card returns to the project already signed in; a protected call such as `GET /v1/Events/List` succeeds; and an expired token sends the user back to the portal and returns them to the same page afterwards.